Systems Programming Knowledge

수식	변수 의미	조건	챕터
`pnb = pending & ~blocked`	pending: 전송된 시그널 비트벡터, blocked: 차단 비트벡터	pnb ≠ 0이면 처리	Ch10
$O_k = \max(P_i, i \le k) / H_k$	P_i: k번째 요청 시점의 페이로드 합계, H_k: k번째 요청 후 힙 크기	$0 < O_k \le 1$, 1에 가까울수록 효율적	Ch7
`D(Rb, Ri, S)` → `Mem[Reg[Rb] + S×Reg[Ri] + D]`	Rb: 베이스 레지스터, Ri: 인덱스(%rsp 불가), S: 스케일(1/2/4/8), D: 변위(상수)	S ∈ {1,2,4,8}	Ch2
`VA = VPN|VPO` → `PA = PPN|PPO`	VPN: 가상 페이지 번호, VPO=PPO: 페이지 오프셋(하위 p비트)	PPO = VPO (동일)	Ch6
단일 PT 크기 = $2^{n-p} \times 2^3$ bytes	n: 가상 주소 비트 수, p: 페이지 크기 비트 수	n=48, p=12 → 2^{39} = 512GB	Ch6
`pushq` = `%rsp -= 8` → store	%rsp: 스택 포인터	스택은 낮은 주소 방향으로 성장	Ch2
`popq` = load → `%rsp += 8`	%rsp: 스택 포인터		Ch2
Timing 공격 복잡도: 52^9 → 52×9	52: 알파벳 대소문자 수, 9: 비밀번호 길이	strcmp 조기 반환 이용	Ch15
Rowhammer: `VA 하위 12bit = PA 하위 12bit`	4KB 페이지 내 오프셋은 VA=PA	인접 Row 주소 추론에 활용	Ch15

예외(Exception) 분류 4가지 ★

종류	발생 원인	동기/비동기	의도성	복귀 위치	예시
Interrupt	외부 장치	비동기	-	I_next	타이머, Ctrl-C, NIC
Trap	명령어 실행	동기	의도적	I_next	syscall, gdb breakpoint
Fault	명령어 실행	동기	비의도	I_current or Abort	Page Fault, Protection Fault 불법 명령

Abort	명령어 실행	동기	비의도	Abort	어, Parity Error

레지스터 저장 규칙 ★

구분	레지스터	저장 책임	동작
Caller-saved	%rax, %rcx, %rdx, %rsi, %rdi, %r8~%r11	호출자(Caller)	Callee가 자유롭게 덮어씀. 필요 시 caller가 call 전에 저장
Callee-saved	%rbx, %r12~%r14, %rbp	피호출자(Callee)	사용 전 push, 반환 전 pop 필수
특수	%rsp	별도 규칙	항상 스택 top 유지

인자 전달 레지스터 순서 ★

순서	**1**	**2**	**3**	**4**	**5**	**6**	**7번째~**	**반환값**
레지스터	%rdi	%rsi	%rdx	%rcx	%r8	%r9	스택(역순 push)	%rax

프로세스 vs 스레드 ★

항목	프로세스	스레드
주소 공간	독립	공유 (힙·코드·데이터·라이브러리)
생성 비용	~20K cycles	~10K cycles
통신 방법	IPC (파이프/소켓/공유메모리)	공유 메모리 직접 접근
스택 보호	VM으로 격리	보호 없음 (포인터로 타 스레드 스택 접근 가능)
계층 구조	트리 (parent-child)	피어(peer) 풀
전용 자원	주소공간, fd, 레지스터, 스택	레지스터, TID, 스택 (보호 없음)

주요 시그널 ★

ID	이름	기본 동작	발생 원인	재정의 가능
2	SIGINT	종료	Ctrl-C	✓
9	SIGKILL	종료	강제 종료	✗
11	SIGSEGV	종료	Segmentation Fault	✓
14	SIGALRM	종료	타이머 만료	✓
17	SIGCHLD	무시	자식 프로세스 종료/정지	✓
20	SIGTSTP	정지	Ctrl-Z	✓

System Call 주요 번호 ★

번호	이름	설명
0	read	파일 읽기
1	write	파일 쓰기
2	open	파일 열기
3	close	파일 닫기
57	fork	프로세스 생성
59	execve	프로그램 실행
60	_exit	프로세스 종료
62	kill	시그널 전송

malloc 가용 블록 추적 방법 비교 ★

방법	구조	할당 시간	삽입 정책	특징
Implicit List	헤더 size로 전체 순회	O(전체 블록)	-	구현 단순, 현대 malloc엔 미사용
Explicit List	free 블록에 prev/next	O(가용 블록)	LIFO/FIFO/주소순	빠름, 최소 블록 크기↑
Segregated List	크기 클래스별 리스트	O(1) 근사	first-fit	처리량↑, 단편화↓, first-fit≈best-fit

Stack Buffer Overflow 공격-방어 ★

공격 기법	방어 기법	원리
Code Injection	NX Bit (XD bit)	스택을 non-executable로 마킹
NX 우회	ROP	기존 코드의 Gadget(ret 끝) 조합
ROP/Code Injection	ASLR	실행마다 주소 랜덤화 → 예측 불가
모든 스택 오버플로우	Stack Canary	%fs:0x28 랜덤값 삽입 → 반환 전 검사
직접	안전한 코딩	gets→fgets, strcpy→strncpy

I/O 함수 계층 선택 가이드 ★

상황	권장	이유
디스크/터미널 파일	Standard I/O (fopen, fprintf 등)	버퍼링, Short count 자동 처리
시그널 핸들러 내부	Unix I/O (read/write)	async-signal-safe 보장
네트워크 소켓	RIO	Short count 처리 + 소켓 호환

최고 성능 필요	Unix I/O	시스템 콜 직접 호출, 최소 오버헤드
Standard I/O + 소켓	✖ 사용 금지	스트림/소켓 제약 충돌

GC 알고리즘 비교

항목	Reference Counting	Mark & Sweep
해제 시점	count = 0 즉시	Stop-the-world 후 일괄
순환 참조	✖ 처리 불가	✓ 처리 가능
시간 오버헤드	중간 (count 관리)	높음/불규칙
공간 오버헤드	count 저장 공간	마크 비트 공간
사용 예	Chrome/Firefox C++	Java, Python, Go

Memory Consistency Model 비교

모델	채택	특징	허용 재정렬
Sequential Consistency	(이론)	가장 강력, 직관적	없음
TSO (Total Store Ordering)	x86	store buffer 허용	Store→Load
Weak Ordering	ARM	거의 모든 재정렬 허용	대부분

Breakpoint 3종류 비교 ★

종류	구현 방식	개수 제한	코드 수정	용도
Software	`int 3` (0xcc) 삽입 → SIGTRAP	무제한	필요 (PTRACE_POKETEXT)	실행 흐름 중단
Hardware	CPU DR0~DR3 레지스터	최대 4개	불필요	실행 흐름 중단
Memory	페이지 권한 변경 → Protection Fault	제한적	불필요	메모리 접근(읽기/쓰기) 감지

ptrace 주요 Request 명령 ★

Request	동작	사용 시점
PTRACE_TRACEME	자신을 부모가 trace하도록 허용	자식 프로세스 초기화 시
PTRACE_PEEKTEXT/DATA	Tracee 메모리 word 읽기	코드/데이터 확인
PTRACE_POKETEXT/DATA	Tracee 메모리 word 쓰기	브레이크포인트(0xcc) 삽입

PTRACE_GETREGS	Tracee 레지스터 전체 복사	중단 시 레지스터 값 읽기
PTRACE_SETREGS	Tracee 레지스터 전체 덮어쓰기	레지스터 값 조작
PTRACE_CONT	정지된 tracee 재개	브레이크포인트 후 계속 실행
PTRACE_SYSCALL	다음 syscall 진입/탈출 시 정지	strace 구현
PTRACE_SINGLESTEP	명령어 1개 실행 후 정지	stepi 구현

Security Attacks 비교 ★

공격	대상	원리	SW 버그 필요	대응책
Cold Boot	DRAM 암호화 키	냉각(-50°C)으로 데이터 소멸 지연	✕ 물리 공격	TRESOR
Timing Side-Channel	비밀 정보	실행 시간 차이 측정	✓ 구현 결함	Constant-time 구현
Cache Side-Channel	메모리 접근 패턴	Flush+Reload 시간 측정	✕ HW 특성	캐시 파티셔닝
Meltdown	커널 메모리	Speculative Exec + 캐시 채널	✕ HW 설계 결함	KPTI
Rowhammer	DRAM 비트	인접 Row 반복 접근 → 비트 플립	✕ HW 물리 결함	ECC DRAM, TRR

Meltdown vs Rowhammer ★

항목	Meltdown	Rowhammer
공격 레이어	CPU (투기적 실행)	DRAM (물리적 비트 플립)
정보 유출 방식	커널 메모리 읽기	PTE 변조 → 권한 상승
캐시 활용	Flush+Reload (covert channel)	clflush (cache bypass)
대응	KPTI (SW 패치, 5~20% 저하)	ECC DRAM (HW 교체)

용어	정의
ASLR	Address Space Layout Randomization. 실행마다 스택/힙/라이브러리 주소 랜덤화. ROP/Code Injection의 주소 예측 방해
async-signal-safe	시그널 핸들러 내에서 안전하게 호출 가능한 함수. 재진입 가능하거나 시그널에 중단되지 않음 (write, kill, _exit ✓ / printf, malloc ✗)
Blocked (시그널)	커널이 프로세스에 전달하지 않도록 억제된 시그널 상태. sigprocmask로 설정

Breakpoint	프로그램 실행을 특정 지점에서 중단시키는 디버깅 메커니즘. Software/Hardware/Memory 3종류
brk 포인터	힙(Heap)의 끝을 가리키는 포인터. sbrk(n)으로 n바이트 증가시켜 힙 확장
Cache Line	캐시와 메모리 사이 데이터 이동의 최소 단위. x86-64에서 64bytes
Cache Side-Channel	캐시 hit/miss 접근 시간 차이를 이용해 피해자의 메모리 접근 패턴을 추론하는 공격
Callee-saved 레지스터	피호출 함수가 사용 전 저장하고 반환 전 복원해야 하는 레지스터. %rbx, %r12~%r14, %rbp
Caller-saved 레지스터	호출자가 필요 시 저장해야 하는 레지스터. Callee가 자유롭게 덮어씀. %rax, %rcx, %rdx, %rsi, %rdi, %r8~%r11
clflush	x86 캐시 라인 강제 비우기 명령어. Rowhammer의 cache bypass와 Flush+Reload 공격에 활용
Coalescing	해제된 인접 가용 블록들을 하나로 합치는 동작. 외부 단편화 방지. Immediate/Deferred 정책
Cold Boot Attack	DRAM을 냉각(-50°C)하여 데이터 소멸을 지연시킨 후 DIMM을 분리해 다른 머신에서 암호화 키를 추출하는 물리 공격
COW (Copy-on-Write)	쓰기 발생 전까지 물리 페이지를 공유. 쓰기 시 Protection Fault → 새 페이지 복사. fork() 성능 최적화의 핵심
CR3 레지스터	현재 프로세스의 L1 페이지 테이블 물리 주소를 저장. 컨텍스트 스위치 시 교체됨
Critical Section	공유 자원에 접근하는 코드 구간. 동시에 하나의 스레드만 실행되어야 함
CSE	Common Subexpression Elimination. 동일한 부분식을 한 번만 계산하는 컴파일러 최적화
Dangling Pointer	이미 해제된 메모리를 가리키는 포인터. Use-After-Free 취약점의 원인
Deadlock	둘 이상의 프로세스/스레드가 서로 상대방이 보유한 자원을 무한히 기다리는 상태
Double-Free	이미 해제된 메모리를 다시 free()하는 버그. 가용 리스트 오염 및 동일 블록 이중 반환 유발
DR0~DR3	x86-64 CPU의 하드웨어 디버그 레지스터. 최대 4개의 Hardware Breakpoint 주소 저장. DR7로 활성화 제어
dup2(oldfd, newfd)	newfd를 oldfd의 복사본으로 만드는 시스템 콜. newfd가 열려있으면 먼저 닫음. I/O 리다이렉션 구현에 사용
ECC DRAM	Error Correcting Code 내장 DRAM. 단일 비트 플립 감지·수정. Rowhammer 방어에 효과적
Exception Table	예외 번호(k)를 인덱스로 핸들러 주소를 저장하는 OS 구성 테이블. 인터럽트 벡터라고도 함
execve()	현재 프로세스에 새 프로그램 로드·실행. PID 유지, 열린 fd 유지, 메모리 교체. 성공 시 반환 없음
External Fragmentation	가용 메모리 합계는 충분하나 연속된 단일 블록이 없어 할당 실패하는 단편화
File Descriptor (fd)	열린 파일을 식별하는 작은 정수. 프로세스별 Descriptor Table에서 관리. 기본: 0(stdin),

	1(stdout), 2(stderr)
Flush+Reload	공유 메모리 캐시를 clflush로 비운 뒤 victim 접근 후 재접근 시간을 측정하는 cache side-channel 기법
fork()	부모 프로세스의 복사본인 자식 프로세스 생성. 1번 호출 2번 반환. 부모→자식PID, 자식→0
Gadget (ROP)	ret 명령어(0xc3)로 끝나는 기존 코드의 명령어 시퀀스. 여러 Gadget을 스택에 연결해 임의 동작 수행
Heap Overflow	힙 버퍼 경계를 넘어 인접 메타데이터(prev/next 포인터)를 덮어쓰는 취약점. Arbitrary Write로 이어짐
Heap Spray	힙의 특정 위치에 원하는 데이터를 대량으로 채워넣는 기법. UAF 익스플로잇에서 vtable 조작에 사용
Implicit Blocking	시그널 핸들러 실행 중 동일 타입 시그널이 자동으로 차단되는 메커니즘. 핸들러 재진입 방지
int 3 (0xcc)	x86 소프트웨어 브레이크포인트 명령어 (1바이트). 실행 시 SIGTRAP 발생. GDB가 PTRACE_POKETEXT로 삽입
Internal Fragmentation	블록 크기가 실제 페이로드보다 커서 낭비되는 내부 공간. 정렬 패딩과 헤더 오버헤드가 원인
ISA	Instruction Set Architecture. 프로세서 명령어 집합과 프로그래머에게 보이는 상태(레지스터, 메모리 등) 정의
KPTI	Kernel Page Table Isolation. Meltdown 대응책. 커널/유저 모드 페이지 테이블 완전 분리. 컨텍스트 스위치 시 5~20% 성능 저하
leaq	Load Effective Address Quadword. 메모리를 읽지 않고 주소값 자체를 목적지 레지스터에 저장. 곱셈 최적화에도 활용
Locality (지역성)	프로그램이 최근(시간적) 또는 인접(공간적) 주소를 재사용하는 경향. 캐시 계층의 효율성 근거
Mark & Sweep	GC 알고리즘. Root에서 DFS로 도달 가능한 블록 마크 후 미마크 블록을 가용 리스트에 추가. 순환 참조 처리 가능
Meltdown	CPU 투기적 실행(Speculative Execution) + 캐시 covert-channel을 결합하여 커널 메모리를 유출하는 HW 설계 결함 공격
MMU	Memory Management Unit. 가상 주소를 물리 주소로 변환하는 하드웨어 유닛. TLB 포함
mmap()	파일이나 익명 메모리를 프로세스 가상 주소 공간에 매핑하는 시스템 콜. MAP_SHARED/MAP_PRIVATE/MAP_ANON
Mutex	Mutual Exclusion. 임계 구역 상호 배제를 보장하는 동기화 도구. test_and_set 원자적 연산으로 구현
NX Bit	No-Execute. 메모리 영역의 실행 권한을 제거하여 Code Injection 방어. PTE의 XD(eXecute Disable) 비트
Open File Table	모든 프로세스가 공유하는 테이블. 각 항목에 file pos(현재 위치)와 refcnt(참조 횟수) 저장
Orphan Process	부모가 먼저 종료된 자식 프로세스. init(PID=1)이 자동으로 입양하여 reaping

Page Fault	접근한 가상 페이지가 물리 메모리에 없을 때 발생하는 Fault 예외. 커널이 디스크에서 페이지 로드 후 재실행
Page Table	VPN(Virtual Page Number)을 PPN(Physical Page Number)으로 매핑하는 커널 자료구조. 프로세스마다 독립적
Pending (시그널)	전송됐지만 아직 수신(처리)되지 않은 시그널 상태. 같은 타입은 최대 1개만 pending (큐 없음)
Process	실행 중인 프로그램의 인스턴스. Logical Control Flow + Private Address Space 두 가지 핵심 추상화 제공
Progress Graph	두 스레드의 실행 상태 공간을 2D로 표현한 그래프. 축=명령어 진행, 점=실행 상태. Race Condition 분석 도구
PTE	Page Table Entry. VPN→PPN 매핑 + 권한 비트(P, R/W, U/S, D, A, XD) 포함. Core i7은 64bit PTE
ptrace	한 프로세스(tracer)가 다른 프로세스(tracee)의 메모리/레지스터를 관찰·제어하는 시스템 콜. GDB·strace의 핵심
Race Condition	공유 자원에 대한 비원자적 접근으로 결과가 스레드 실행 순서에 따라 달라지는 버그
Reentrant Function	공유 변수(전역/정적)를 사용하지 않아 동기화 없이 여러 스레드에서 동시 호출 가능한 함수. Thread-safe의 부분집합
ROP	Return-Oriented Programming. 기존 코드의 Gadget(ret으로 끝나는 명령 시퀀스)을 조합하여 NX Bit를 우회하는 공격
Rowhammer	DRAM의 인접 Row를 반복 접근(hammer)하여 전하 누설로 victim row의 비트를 뒤집는 하드웨어 공격
Semaphore	음수가 될 수 없는 정수형 동기화 변수. wait(P: s–)와 post(V: s++) 원자적 연산으로 조작. 불변: s≥0
Short Count	read/write 시 요청한 바이트보다 적게 처리되는 정상적 현상. EOF/터미널 입력/네트워크 소켓에서 발생. 오류 아님
Side-Channel Attack	알고리즘 취약점이 아닌 구현의 부수 정보(타이밍, 전력 소비, 음향, 전자기 누출)를 이용한 공격
Signal	커널이 프로세스에게 이벤트 발생을 알리는 작은 메시지. ID 1~30. 큐 없음(같은 타입 최대 1개 pending)
Speculative Execution	CPU가 성능을 위해 분기 결과·권한 확인이 완료되기 전에 명령어를 미리 실행하는 기법. Meltdown의 근본 원인
Splitting	가용 블록을 요청 크기만큼만 잘라 할당하고 나머지를 새 가용 블록으로 분리하는 동작
Stack Canary	함수 진입 시 버퍼와 반환 주소 사이에 배치하는 랜덤값(%fs:0x28). 반환 전 값 비교하여 스택 오버플로우 탐지
Starvation	전체 시스템은 진행되나 특정 프로세스/스레드가 필요한 자원을 무한히 얻지 못하는 상태
Strength Reduction	곱셈·나눗셈 등 비용이 큰 연산을 시프트·덧셈 등 저렴한 연산으로 대체하는 컴파일러 최적화

Thread Safety	여러 스레드가 동시에 호출해도 항상 올바른 결과를 반환하는 함수의 속성. Reentrant ⊂ Thread-safe
Thrashing	프로세스의 Working Set이 물리 메모리를 초과하여 페이지 교체(swap)가 폭주하는 현상
TLB	Translation Lookaside Buffer. MMU 내부의 소형 PTE 캐시. Hit 시 메모리 접근 1회로 주소 변환 완료
Tracer / Tracee	ptrace에서 제어하는 프로세스(tracer, 예: GDB)와 제어받는 프로세스(tracee, 예: 디버깅 대상)
Transient Instruction	투기적 실행 중 실행됐다가 커밋되지 않고 되돌려지는 명령어. Meltdown 공격의 핵심 개념
TRESOR	암호화 키와 AES 연산을 CPU 레지스터/캐시에만 유지하여 Cold Boot Attack을 방어하는 Linux 커널 패치
Use-After-Free (UAF)	해제된 메모리를 재사용하는 취약점. Dangling Pointer + Heap Spray → vtable 조작 → 제어 흐름 탈취
v-node Table	파일의 실제 메타데이터(크기, 타입, 접근 권한, inode 등 stat 구조체 내용)를 저장하는 커널 테이블
Virtual Memory	물리 메모리보다 큰 주소 공간 제공 + 프로세스 격리를 구현하는 OS 추상화. 캐싱/관리/보호 3가지 역할
VPN / VPO	Virtual Page Number(상위 비트, 페이지 테이블 인덱스) / Virtual Page Offset(하위 p비트, 페이지 내 위치)
Zombie Process	종료됐지만 부모가 wait()를 호출하지 않아 exit status 등 일부 자원을 점유 중인 프로세스. ps에서 `<defunct>`

Ch1. Introduction

핵심 개념

Five Great Realities: 추상화 아래 숨겨진 현실을 이해해야 진짜 시스템 프로그래머다.

Reality	내용	시험 포인트
#1	Int ≠ Integer, Float ≠ Real → 유한 비트 → 오버플로우, 결합법칙 위반	`50000*50000` 오버플로우, `(1e20+-1e20)+3.14 ≠ 1e20+(-1e20+3.14)`
#2	어셈블리 이해 필수 → 버그 분석, 최적화, 역공학	컴파일러 최적화 결과 해석
#3	C는 메모리 보호 없음 → 배열 경계 초과도 OK	버그 발생 위치 ≠ 오류 관측 위치

#4	성능 ≠ 접근 복잡도 → 메모리 접근 패턴이 결정적	copyij(4.3ms) vs copyji(81.8ms) 19배 차이
#5	I/O·네트워크도 시스템	I/O 성능이 프로그램 신뢰성에 영향

★ **핵심 문제 1**: copyij vs copyji 성능 차이 설명

```
Q. void copyij(int src[2048][2048], int dst[2048][2048])에서
   i가 외부 루프일 때와 j가 외부 루프일 때 성능이 19배 차이 나는 이유는?

A. C 2D 배열은 Row-Major Order 저장:
   메모리: src[0][0], src[0][1], ..., src[0][2047], src[1][0], ...
                                                        ↑ 4×2048 = 8192
bytes 간격

   i 외부(copyij): src[i][j], src[i][j+1] → 연속 주소 → 공간적 지역성 ↑
     → 64-byte Cache Line에 16개 int 포함 → 16번에 1번 Miss
     → Cache Hit Rate 매우 높음 → 빠름

   j 외부(copyji): src[i][j], src[i+1][j] → 8192 bytes 건너뜀
     → 매번 새 Cache Line → 거의 매번 Miss
     → Miss 시 DRAM 접근(~100 cycles) → Cache Hit(~4 cycles)의 25배 비
용
     → 느림
```

Ch2-1. Machine Basics

핵심 개념

ISA vs Microarchitecture: 프로그래머는 ISA(레지스터, 명령어)만 보면 된다. 파이프라인, 캐시, TLB는 보이지 않는다.

AT&T 문법: `op Src, Dest` (Intel과 반대 — 항상 헷갈리는 부분)

Operand 3종과 핵심 제약:

```
Immediate: $0x400, $-533        (값 자체)
Register:  %rax, %r13           (레지스터 값)
Memory:    (%rax), 8(%rcx)      (메모리에서 로드)
⋆ Memory → Memory 직접 이동 불가! 반드시 레지스터 경유
```

메모리 주소 지정 일반형:

```
D(Rb, Ri, S) = Mem[Reg[Rb] + S×Reg[Ri] + D]
  S ∈ {1, 2, 4, 8}    ← 이 4가지만!
  Ri ≠ %rsp           ← %rsp는 인덱스 레지스터 불가
```

leaq vs movq — 핵심 구분:

```
leaq Src, Dst: 주소값을 Dst에 저장  (메모리 접근 없음)
movq Src, Dst: 메모리 값을 Dst에 저장 (메모리 접근 있음)

leaq 8(%rdi), %rax  →  %rax = %rdi + 8      ← 주소
movq 8(%rdi), %rax  →  %rax = Mem[%rdi+8]   ← 메모리의 값

leaq 응용: b*3 최적화
  leaq (%rdi,%rdi,2), %rax  →  %rax = %rdi + 2×%rdi = 3×%rdi
```

★ **핵심 문제 1**: 주소 계산

```
Q. %rdx=0xf000, %rcx=0x0100 일 때 다음 각 표현식의 주소 값은?

(a) 0x8(%rdx)       (b) (%rdx,%rcx)
(c) (%rdx,%rcx,4)   (d) 0x80(,%rdx,2)

풀이:
(a) Mem[0xf000 + 8]            = Mem[0xf008]
(b) Mem[0xf000 + 0x100]        = Mem[0xf100]
(c) Mem[0xf000 + 4×0x100]     = Mem[0xf400]
(d) Mem[2×0xf000 + 0x80]      = Mem[0x1e080]
```

★ **핵심 문제 2**: leaq를 이용한 산술 표현

```
Q. 다음 C 코드를 컴파일하면 어떤 어셈블리가 나오는가?
   long scale(long x, long y, long z) {
       return 5*x + 2*y + 8*z;
   }
   인자: %rdi=x, %rsi=y, %rdx=z

풀이 (leaq 활용):
   leaq (%rdi,%rdi,4), %rax   # %rax = x + 4x = 5x
   leaq (%rsi,%rsi), %rcx     # %rcx = y + y = 2y  (or addq
%rsi,%rsi)
   leaq (,%rdx,8), %rdx       # %rdx = 8z
   addq %rcx, %rax            # %rax = 5x + 2y
   addq %rdx, %rax            # %rax = 5x + 2y + 8z
   ret
```

Ch2-2. Machine Control

핵심 개념

Condition Codes (4개의 1-bit 플래그):

```
CF: unsigned 오버플로우 (Carry/Borrow 발생)
ZF: 결과 = 0
SF: 결과 < 0 (최상위 비트 = 1)
OF: signed 오버플로우 (부호 바뀜)

설정 방법:
  암묵적: addq, subq, imulq 등 산술연산 (단, leaq는 설정 안함!)
  cmpq b, a: a-b 계산 (저장 안함) → ★ 순서 주의: cmpq Src2, Src1 →
Src1-Src2
  testq b, a: a&b 계산 (저장 안함) → 주로 testq %rax,%rax (0 확인)
```

모든 루프는 결국 **goto + 조건 점프**로 변환된다:

```
while: Jump-to-Middle    →  goto test; loop: body; test: if(c) goto
loop;
do-while:                →  loop: body; if(c) goto loop;
for:                     →  init; goto test; loop: body; update;
test: if(c) goto loop;
```

★ **핵심 문제 1**: Condition Code 추론

```
Q. 다음 어셈블리 실행 후 각 CC의 값은? (a = 0x8000000000000000, b =
0x8000000000000000)
   addq %rsi, %rdi   # %rdi = a + b (signed 관점에서는 음수 + 음수 = 양
```

```
수?)

풀이:
   a + b = 0x8000000000000000 + 0x8000000000000000 =
0x10000000000000000
   64-bit로 자르면: 0x0000000000000000

   CF = 1 (최상위 비트에서 carry 발생 → unsigned overflow)
   ZF = 1 (결과 = 0)
   SF = 0 (결과의 최상위 비트 = 0)
   OF = 1 (음수 + 음수 = 양수 → signed overflow)
```

★ 핵심 문제 2: 어셈블리 → C 복원

```
Q. 다음 어셈블리를 C 코드로 복원하라.
   # %rdi = x, %rsi = y, 반환값 = %rax
   absdiff:
     cmpq %rsi, %rdi
     jle  .L4
     movq %rdi, %rax
     subq %rsi, %rax
     ret
   .L4:
     movq %rsi, %rax
     subq %rdi, %rax
     ret

풀이:
   cmpq %rsi, %rdi  →  %rdi - %rsi 비교  →  if (x <= y) goto .L4
   then: %rax = %rdi - %rsi = x - y
   else: %rax = %rsi - %rdi = y - x

   long absdiff(long x, long y) {
       if (x > y) return x - y;
       else       return y - x;
   }
```

Ch2-3. Procedures

핵심 개념

```
스택 성장 방향: 높은 주소 → 낮은 주소
pushq reg: %rsp -= 8, Mem[%rsp] = reg
popq  reg: reg = Mem[%rsp], %rsp += 8

call label: push 복귀주소(%rip_next), jmp label
ret:        pop %tmp, jmp *%tmp
```

인자 전달 순서 (**반드시 암기**):

```
1번째: %rdi   2번째: %rsi   3번째: %rdx
4번째: %rcx   5번째: %r8    6번째: %r9
7번째~: 스택 (역순 push)
반환값: %rax
```

레지스터 저장 규칙:

```
Caller-saved: %rax, %rcx, %rdx, %rsi, %rdi, %r8~%r11
  → Callee가 자유롭게 덮어씀. Caller가 필요하면 call 전에 저장
```

```
Callee-saved: %rbx, %r12, %r13, %r14, %rbp
  → Callee가 사용 전 push, 반환 전 pop 필수

★ 핵심: Callee-saved를 쓰는 이유
   = Caller가 call 이후에도 해당 레지스터 값이 필요할 때
```

스택 프레임 레이아웃:

```
높은 주소
┌──────────────────────┐
│  Caller 인자 7+       │ (7개 이상일 때)
├──────────────────────┤
│  반환 주소 (8 bytes)  │ ← call 명령어가 push
├──────────────────────┤ ← (선택) %rbp
│  Callee-saved 레지스터│ ← push %rbx 등
├──────────────────────┤
│  지역 변수            │
├──────────────────────┤
│  인자 빌드 공간       │ (다음 호출용)
└──────────────────────┘ ← %rsp
낮은 주소
```

★ 핵심 문제 1: 스택 프레임 추적

```
Q. 다음 코드에서 함수 진입 직후 스택 상태를 그려라.
   long sum(long a, long b) { return a + b; }
   main에서: long r = sum(3, 7);

풀이:
  caller (main):
    movq $3, %rdi        # a = 3 (1번째 인자)
    movq $7, %rsi        # b = 7 (2번째 인자)
    call sum             # push 복귀주소, jmp sum

  sum 진입 직후 스택:
  높은 주소
  ┌──────────────────────┐
  │ 복귀주소 (8 bytes)   │ ← %rsp
  └──────────────────────┘
  낮은 주소

  sum 실행:
    %rax = %rdi + %rsi = 3 + 7 = 10
    ret  # pop 복귀주소 → %rip, %rsp += 8
```

★ 핵심 문제 2: Callee-saved 필요성

```
Q. 왜 pcount 재귀 함수에서 %rbx를 사용해 x & 1을 저장하는가?
   long pcount(unsigned long x) {
       if (x == 0) return 0;
       return (x & 1) + pcount(x >> 1);
   }

풀이:
  pcount는 x & 1 값을 재귀 호출 이후에도 사용해야 함.
  재귀 call 시 %rdi(%rax 등)는 Caller-saved → 덮어써짐.
  따라서 x & 1을 Callee-saved인 %rbx에 저장:
    pushq %rbx          # %rbx 보존
    movq  %rdi, %rbx   # x를 %rbx에 백업
    ...
    call  pcount        # %rbx는 보존됨 (callee-saved)
```

```
    andl  $1, %ebx     # x & 1 복원
    addq  %rbx, %rax   # (x&1) + 재귀 결과
    popq  %rbx          # %rbx 복원
    ret
```

Ch3. Stack Buffer Overflow

핵심 개념

C는 배열 경계를 검사하지 않는다. gets(), strcpy() 등 길이 무제한 함수로 스택 버퍼를 초과하면 반환 주소까지 덮어쓸 수 있다.

취약점 구조:

```
char name[32]이 rbp-0x20에 위치할 때:

높은 주소
┌──────────────────┐
│  반환 주소        │ ← name + 40 (name[40]~name[47])
├──────────────────┤
│  saved %rbp       │ ← name + 32 (name[32]~name[39])
├──────────────────┤ ← %rbp
│  name[0..31]      │ ← %rbp - 0x20
└──────────────────┘ ← %rsp
낮은 주소

⋆ 41번째 바이트(name[40])부터 반환 주소 덮어씀
  → ret 실행 시 공격자가 원하는 주소로 점프
```

공격-방어 체계:

```
Code Injection → NX Bit로 방어 (스택 non-executable)
NX 우회       → ROP (기존 코드 Gadget 조합, 0xc3=ret으로 끝나는 시퀀스)
ROP           → ASLR로 주소 예측 어렵게
오버플로우 탐지→ Stack Canary (%fs:0x28 랜덤값, 반환 전 검사)
```

★ 핵심 문제 1: 오버플로우 offset 계산

```
Q. char buf[32]가 rbp-0x20에 위치한다. 반환 주소를 덮어쓰려면
   gets(buf)에 최소 몇 바이트를 입력해야 하는가?
   (주어진 정보: saved rbp = 8 bytes, 반환 주소 = 8 bytes)

풀이:
  buf 시작: rbp - 0x20 = rbp - 32
  saved rbp: rbp + 0  (= buf + 32)
  반환 주소: rbp + 8  (= buf + 40)

  반환 주소를 덮어쓰려면 buf[40]~buf[47]에 쓰기 필요
  → 최소 41 bytes 필요 (buf[0]~buf[39]: 패딩 40 bytes, buf[40]~: 반환주소)

  정확히 덮어쓰려면: 40 bytes 패딩 + 8 bytes 주소 = 48 bytes 입력
```

★ 핵심 문제 2: ROP chain 이해

```
Q. NX bit가 켜져 있을 때 공격자는 어떻게 임의 코드를 실행하는가?

A. Return-Oriented Programming (ROP):
   기존 .text 섹션의 코드를 활용 → NX와 무관
```

```
Gadget: ret(0xc3)으로 끝나는 기존 명령어 시퀀스
예:
  0x4005a1: pop %rdi ; ret   ← Gadget 1
  0x4005b3: pop %rsi ; ret   ← Gadget 2

스택에 Gadget 주소 체인 배치:
┌──────────────────┐
│ Gadget1 주소     │ ← 반환 주소로 덮어씀
│ 인자값 (0x1234)  │ ← pop %rdi가 가져갈 값
│ Gadget2 주소     │ ← Gadget1의 ret가 여기로 점프
│ 인자값 (0x5678)  │
│ syscall 주소     │
└──────────────────┘

실행 흐름: ret → Gadget1 → ret → Gadget2 → ret → syscall
→ 체인처럼 연결하여 임의 동작 수행
→ .text에 있는 코드이므로 NX bit 우회 가능!
```

★ 핵심 문제 3: Stack Canary 동작

```
Q. Stack Canary가 buf[32] 위에 배치될 때, 공격자가 반환 주소를 덮어쓰면
   어떤 일이 일어나는가?

A. 스택 레이아웃 (canary 포함):
   ┌──────────────────┐
   │  반환 주소        │
   ├──────────────────┤
   │  saved %rbp      │
   ├──────────────────┤
   │  CANARY (8 bytes)│ ← %fs:0x28의 값
   ├──────────────────┤
   │  buf[0..31]      │
   └──────────────────┘

   반환 주소까지 덮어쓰려면 반드시 canary도 덮어씀
   함수 반환 전: mov -0x8(%rbp), %rax; xor %fs:0x28, %rax
   → canary가 변조됐으면 ZF=0 → jne __stack_chk_fail
   → 프로그램 즉시 종료 (abort)

   결론: Canary 덮어쓰지 않고 반환 주소만 바꾸는 것은 불가능
   (일부 취약점: 특정 바이트만 덮어쓰는 형식 스트링 공격 등)
```

Ch4. Memory Hierarchy

핵심 개념

CPU-Memory Gap: CPU는 빠르고 DRAM은 느리다(약 100배). 이 격차를 지역성으로 해결한다.

지역성 원칙:

```
시간적 지역성: 최근 접근한 데이터는 곧 다시 접근
공간적 지역성: 최근 접근한 위치 근처를 곧 접근

캐시가 동작하는 이유: 프로그램이 이 패턴을 따르기 때문
```

캐시 동작:

```
Cache Line = 64 bytes (블록 전송 단위)
Hit:  요청 블록이 캐시에 있음 → 빠름 (~4 cycles)
```

```
Miss: 요청 블록이 없음 → 다음 레벨 메모리에서 로드 (~100 cycles for DRAM)
      Placement Policy: 어디에 배치할지
      Replacement Policy: 무엇을 교체할지 (LRU, FIFO, Random)
```

★ 핵심 문제 1: 지역성 분석

```
Q. 다음 두 함수 중 어떤 것이 더 좋은 캐시 성능을 갖는가? 이유를 설명하라.

// 함수 A:
for (i = 0; i < M; i++)
    for (j = 0; j < N; j++)
        sum += a[i][j];

// 함수 B:
for (j = 0; j < N; j++)
    for (i = 0; i < M; i++)
        sum += a[i][j];

A. 함수 A가 더 좋은 캐시 성능을 가진다.

근거:
  C 2D 배열: Row-Major Order → a[i][0], a[i][1], ..., a[i][N-1],
a[i+1][0], ...

  함수 A(행 우선): a[i][0]→a[i][1]→...  연속 주소 → 공간적 지역성 ↑
    Cache Line 64 bytes = int 16개 → 16번에 1번 Miss

  함수 B(열 우선): a[0][j]→a[1][j]→...  N×4 bytes 건너뜀
    N=2048이면 8192 bytes 건너뜀 → 매번 새 Cache Line → 거의 매번 Miss

  성능 차이: 최대 16배 이상 (Cache Miss 비율 차이)
```

★ 핵심 문제 2: Cache Miss 유형 분류

```
Q. 3C 모델로 캐시 미스를 분류하라.

A. 3C (Three C's of Cache Misses):

1. Compulsory Miss (Cold Miss):
   = 처음 접근하는 블록 → 반드시 발생, 피할 수 없음
   → Prefetching으로 완화 가능

2. Capacity Miss:
   = Working Set > Cache 크기 → 데이터가 캐시에 들어가지 않음
   → 더 큰 캐시 필요, 또는 Working Set 줄이기

3. Conflict Miss:
   = Set-associative 캐시에서 동일 Set에 너무 많은 블록이 매핑
   → Associativity 높이기, 접근 패턴 변경

예: Direct-mapped cache (size=32KB, line=64B, sets=512)
   stride=512*64=32KB인 접근 → 모두 Set 0으로 매핑 → Conflict Miss 폭발
```

Ch5. Code Optimization & Linking

핵심 개념

컴파일러 최적화 원칙: 프로그램 동작이 바뀌지 않는 범위에서만 최적화

컴파일러가 **할 수 없는** 최적화:

```
1. Side effect 있는 함수를 루프 밖으로 이동 불가
   → strlen(s)는 s를 수정할 수 있으므로 Code Motion 불가

2. Signed integer overflow = Undefined Behavior 이용한 최적화
   → 컴파일러가 "항상 true"인 조건을 제거할 수 있음

3. 함수 단위 분석 → aliasing 제거 불가
   → *p와 *q가 같은 주소를 가리킬 수 있으면 재정렬 불가
```

링킹 3단계:

```
1. 심볼 테이블 구성 (어셈블러):
   Global: non-static 함수/전역 → 다른 파일에서 참조 가능
   Local:  static 함수/변수    → 파일 내부에서만
   External Ref: 다른 파일의 심볼 참조

2. 심볼 해석 (링커): External Ref → 정의에 연결

3. 재배치 (링커): .o의 임시주소(0x0) → 최종 절대주소
   (objdump -r -d로 재배치 엔트리 확인)
```

★ 핵심 문제 1: 최적화 한계 분석

```
Q. 다음 코드에서 컴파일러가 lower1을 lower2로 최적화할 수 없는 이유는?

void lower1(char *s) {
    for (int i = 0; i < strlen(s); i++)
        if (s[i] >= 'A' && s[i] <= 'Z') s[i] -= 32;
}

void lower2(char *s) {
    int n = strlen(s);  // 루프 밖으로 이동
    for (int i = 0; i < n; i++)
        if (s[i] >= 'A' && s[i] <= 'Z') s[i] -= 32;
}

A. 컴파일러는 s[i] -= 32가 s 문자열 자체를 수정함을 인식.
   strlen(s)는 s를 읽으므로, s가 변경되면 strlen 결과도 달라질 수 있음.
   → 컴파일러는 side effect 가능성 때문에 안전하게 매 반복 strlen 호출 유지
   → lower1: O(n²), lower2: O(n) (프로그래머가 직접 최적화해야 함)
```

★ 핵심 문제 2: 링킹 순서와 심볼 해석

```
Q. main.c가 sum()을 호출하고 sum.c에 정의된 경우:
   (1) 컴파일 단계에서 sum은 어떻게 처리되는가?
   (2) 링킹 후 실행 파일에서 call sum은 어떻게 변하는가?

A.
(1) 컴파일 단계 (main.o):
    call 0x0000 ; R_X86_64_PC32 sum  ← 임시 주소 + 재배치 엔트리
    "sum은 external reference이므로 주소를 나중에 채울 것"

(2) 링킹 후:
    sum()이 0x4004e8에 배치됐다면:
    call 0x4004e8  ← 실제 절대 주소로 패치됨
    (PC-relative addressing: 호출 지점 0x4004de에서 +0xa = 0x4004e8)
```

Ch6. Virtual Memory

핵심 개념

VM의 3역할:

```
1. Caching: DRAM이 디스크의 캐시 역할
   → 모든 데이터는 원래 디스크에 있음
   → 자주 접근하는 4KB 페이지를 DRAM에 캐시

2. Memory Management: 프로세스마다 독립적인 선형 주소 공간
   → fork() 시 COW로 효율적 복제

3. Memory Protection: PTE 권한 비트(R/W, U/S, XD)로 접근 제어
   → 매 메모리 접근 시 MMU가 검사
```

주소 변환:

```
VA(48-bit) = VPN(36-bit) | VPO(12-bit)
PA         = PPN(40-bit) | PPO(12-bit)  (PPO = VPO)

4단계 PT (Core i7):
  VA = VPN1(9) | VPN2(9) | VPN3(9) | VPN4(9) | VPO(12)
  CR3 → L1 PT → L2 PT → L3 PT → L4 PT → PPN
  L1 PTE가 null이면 → 512GB 영역 전체 unmapped
  → 미사용 영역에 PT 메모리 낭비 없음 (Multi-level의 장점)
```

★ 핵심 문제 1: 단일 PT 크기 계산

```
Q. 48-bit VA, 4KB 페이지, 8-byte PTE인 시스템에서 단일 PT의 크기는?

풀이:
  페이지 크기 = 2^12 bytes
  가상 페이지 수 = 2^48 / 2^12 = 2^36
  단일 PT 크기 = 2^36 × 8 bytes = 2^39 bytes = 512 GB

  → 비현실적! → 4단계 PT로 해결
  4단계 PT: VA = 9+9+9+9+12 bits
  L1 PT 크기 = 2^9 × 8 bytes = 4 KB (단 하나의 페이지만 필요)
```

★ 핵심 문제 2: Page Fault 처리 7단계

```
Q. 프로세스가 가상 주소 A에 접근했을 때 Page Fault가 발생하면 어떤 일이 일어
나는가?

A. Page Fault 처리 7단계:
①  CPU가 VA 생성 → MMU가 페이지 테이블에서 PTE 조회
②  PTE의 valid bit(P) = 0 → Page Fault 예외 발생 (Fault 타입)
③  커널의 Page Fault Handler로 제어 전달
④  Handler가 교체할 Victim 페이지 선택 (LRU 등)
⑤  Victim의 D(Dirty) bit = 1이면 → 디스크에 먼저 write-back
⑥  필요한 페이지를 디스크에서 물리 메모리로 로드
⑦  PTE 갱신 (P=1, PPN 설정) → iret → 원래 명령어 재실행 (이번엔 Hit)

⋆ Fault 타입이므로 I_current(원인 명령어)를 재실행!
   (Interrupt/Trap의 I_next와 다름)
```

★ 핵심 문제 3: TLB Hit vs Miss 비교

```
Q. VA→PA 변환 시 TLB Hit과 TLB Miss의 메모리 접근 횟수를 비교하라.

A.
TLB Hit:
  ① CPU → MMU: VA 전송
  ② MMU: TLB에서 VPN→PPN 변환 (캐시 히트, DRAM 접근 없음)
  ③ MMU → Cache/Memory: PA로 데이터 요청
```

```
  ④ 데이터 반환
  → DRAM 접근: 1회 (데이터만)

TLB Miss:
  ① CPU → MMU: VA 전송
  ② MMU: TLB miss → 페이지 테이블에서 PTE 조회 (DRAM 접근 1회)
     4단계 PT라면 최대 4회 DRAM 접근
  ③ TLB 갱신 후 PA 생성
  ④ MMU → Cache/Memory: PA로 데이터 요청 (DRAM 접근 1회)
  → DRAM 접근: 2~5회

TLB Miss가 드문 이유: 시간적 지역성 (최근 변환한 페이지를 곧 다시 접근)
```

★ 핵심 문제 4: COW (Copy-on-Write) 동작

```
Q. fork() 직후 자식이 지역 변수 x를 수정하면 어떤 일이 일어나는가?

A. fork() 시:
  ① 부모의 페이지 테이블을 자식에게 복사
  ② 두 프로세스의 모든 페이지를 Read-Only로 표시 (COW 설정)
  ③ 물리 페이지는 공유 (아직 복사 안 함)

  자식이 x를 쓰려 하면:
  ④ Read-Only 페이지에 쓰기 → Protection Fault 발생
  ⑤ 커널 핸들러: 해당 물리 페이지를 새 물리 페이지에 복사
  ⑥ 자식의 PTE를 새 물리 페이지로 갱신, R/W 권한 부여
  ⑦ 쓰기 명령어 재실행 → 성공

  결과: 부모의 x와 자식의 x는 이제 서로 다른 물리 페이지
  → 완전히 독립적인 값 유지
```

Ch7. Dynamic Memory Allocation

핵심 개념

```
malloc: x86-64에서 16-byte 정렬 보장
brk 포인터: 힙의 끝을 가리킴 (sbrk()로 증가)

단편화:
  내부(Internal): 블록 > 페이로드 (정렬 패딩, 헤더 오버헤드)
  외부(External): 합계는 충분하나 연속 블록 없음 → Coalescing으로 해결

블록 헤더 트릭:
  16-byte 정렬 → size 하위 4bit = 항상 0
  → alloc bit를 하위 비트에 저장 (공간 절약)
  header = size | alloc  (예: size=32, alloc=1 → 0x21)
```

3가지 가용 블록 추적 방법:

```
Implicit List: 헤더로 전체 순회 → O(전체 블록) → 현대에 미사용
Explicit List: free 블록에 prev/next → O(가용 블록) → 빠름
Segregated:   크기 클래스별 리스트 → O(1) 근사, first-fit ≈ best-fit
```

★ 핵심 문제 1: Coalescing과 False Fragmentation

```
Q. 아래 상태에서 malloc(6*SIZ)를 요청하면 성공하는가?
   (합계 가용 메모리 = 8*SIZ인데도)
   [free:4] [alloc:8] [free:4]  (단위: SIZ)

A. 실패한다. (External Fragmentation)
```

```
   각 가용 블록 크기: 4*SIZ, 4*SIZ
   연속된 가용 블록 없음 → malloc(6*SIZ) 불가

   해결: Coalescing (인접 가용 블록 합병)
   free() 직후 인접 블록이 가용이면 합병:
   [free:4] + [free:4] → [free:8] → malloc(6*SIZ) 성공

   False Fragmentation 발생 조건:
   free() 후 인접 블록이 가용인데 합병하지 않은 경우

   정책:
     Immediate Coalescing: free() 즉시 합병 (단순)
     Deferred Coalescing: 나중에 일괄 합병 (성능 향상 가능)
```

★ 핵심 문제 2: Heap Overflow → Arbitrary Write

```
Q. Heap Overflow가 어떻게 임의 메모리 쓰기(Arbitrary Write)로 이어지는가?

A. K&R malloc의 free list 구조:
   [allocated][free chunk: prev|next][allocated]

   free() 시 인접 블록 합병:
     node->prev->next = node->next;  ← 이 코드가 핵심

   공격자가 allocated 블록을 overflow로 다음 free 블록의 prev/next를 덮어
쓰:
     prev = 공격자 원하는 주소 A
     next = 공격자 원하는 값 B

   free() 호출 시:
     A->next = B  → 주소 A에 값 B를 씀 = Arbitrary Write!

   실제 공격: GOT(Global Offset Table) 주소를 A로,
              shellcode 주소를 B로 설정 → 제어 흐름 탈취
```

★ 핵심 문제 3: Use-After-Free 익스플로잇

```
Q. Use-After-Free가 어떻게 제어 흐름 탈취로 이어지는가?

A. 단계별 공격:
   ① Doc 객체 생성, Body 객체 생성
   ② doc->child = body (포인터 연결)
   ③ delete body  → body 해제 (doc->child는 dangling pointer)
   ④ 공격자가 해제된 메모리 영역에 fake Body 객체 스프레이 (Heap Spray)
      fake Body의 vtable pointer = 공격자 제어 가능
   ⑤ doc->child->getAlign() 호출
      → doc->child (dangling) → fake Body의 vtable 접근
      → 공격자의 코드 실행

   핵심: C++ virtual function은 vtable을 통해 호출
         vtable pointer를 제어하면 = 제어 흐름 탈취
```

Ch8. Exceptions

핵심 개념

4가지 예외 분류 (★ 항상 출제):

```
          | 발생원인      | 동기/비동기 | 의도성    | 복귀 위치
Interrupt | 외부 장치     | 비동기      | -         | I_next
```

```
Trap      | 명령어 실행  | 동기       | 의도적   | I_next
Fault     | 명령어 실행  | 동기       | 비의도   | I_current 재실행 or
Abort
Abort     | 명령어 실행  | 동기       | 비의도   | Abort (종료)
```

System Call 동작:

```
%rax = syscall 번호 (read=0, write=1, open=2, fork=57, execve=59,
_exit=60, kill=62)
%rdi, %rsi, %rdx, %r10, %r8, %r9 = 인자 1~6
syscall 명령어 → Trap → 커널 → 서비스 수행
반환값 → %rax (음수 = 오류)
```

★ 핵심 문제 1: 예외 분류

```
Q. 다음 각 상황을 예외 종류로 분류하고 복귀 위치를 설명하라.

(a) 유저가 Ctrl-C를 누름
(b) movl $0xd, 0x8049d10 실행 시 해당 페이지가 DRAM에 없음
(c) 유저 프로그램이 open("file.txt", O_RDONLY) 호출
(d) CPU가 불법 명령어를 만남

A.
(a) Interrupt (비동기, 외부 장치) → I_next 복귀
    SIGINT 시그널이 해당 프로세스에 전달됨

(b) Page Fault (Fault, 동기, 비의도)
    → 복구 가능: 디스크에서 페이지 로드 후 I_current(movl) 재실행
    → 복구 불가(유효하지 않은 주소): SIGSEGV 전달 후 Abort

(c) System Call (Trap, 동기, 의도적) → I_next 복귀
    %eax = 2(open), syscall 명령어 → 커널이 파일 열기

(d) Abort (동기, 비의도) → 프로그램 종료
```

★ 핵심 문제 2: iret vs ret

```
Q. iret와 ret의 차이는?

A.
ret:  pop %rip    → 이전 코드로 복귀 (유저→유저 또는 커널→커널)

iret: pop %rip    ← 기본 동작은 ret과 같음
      pop CS      ← Code Segment 복원 (Ring level 결정)
      pop RFLAGS  ← 플래그 레지스터 복원
      pop %rsp    ← 스택 포인터 복원 (선택적)

iret의 핵심: 권한 레벨(Ring 0↔Ring 3) 전환
Page Fault 처리 후 iret → 유저 모드로 복귀 + 권한 복원
```

Ch9. Processes

핵심 개념

Process = Logical Control Flow + Private Address Space (2개의 핵심 추상화)

```
fork(): 1번 호출 2번 반환
  → 부모: 자식 PID (양수)
  → 자식: 0
```

```
  → 실행 순서: 비결정적!
  → 메모리: COW로 물리 페이지 공유 (쓰기 시 복사)

execve(): 1번 호출 정상 반환 없음
  → PID 유지, 열린 fd 유지 (O_CLOEXEC 제외)
  → 코드/데이터/스택 교체

wait(): 자식 종료 대기 + Zombie 회수
zombie: 종료됐지만 wait()를 받지 못한 자식 (<defunct>)
orphan: 부모보다 먼저 종료 → init(PID=1)이 입양하여 reaping
```

★ 핵심 문제 1: fork() 출력 분석

```
Q. 다음 코드의 가능한/불가능한 출력을 판별하라.
   int main() {
       printf("L0\n"); fork();
       printf("L1\n"); fork();
       printf("Bye\n");
   }

A. Process Graph 분석:
   fork() 후 두 흐름 각각 L1, Bye 출력
   두 번째 fork() 후 또 두 흐름 각각 Bye 출력

   총 출력: L0×1, L1×2, Bye×4

   가능한 순서 조건:
   - L0은 반드시 모든 출력 중 가장 먼저 (fork 전에 출력됨)
   - 각 프로세스 내에서는 L1이 Bye보다 먼저
   - 다른 프로세스 간 순서는 비결정적

   ✓ L0 L1 Bye L1 Bye Bye Bye   (가능)
   ✓ L0 L1 L1 Bye Bye Bye Bye   (가능)
   ✗ L1 L0 Bye Bye L1 Bye Bye   (불가: L0이 먼저여야 함)
   ✗ L0 Bye L1 ...               (불가: 같은 프로세스에서 L1이 Bye 전이어야 함)
```

★ 핵심 문제 2: Zombie vs Orphan

```
Q. Zombie와 Orphan Process를 구분하고 각각의 문제점을 설명하라.

A.
Zombie Process:
  = 자식이 종료됐지만 부모가 wait()를 호출하지 않은 상태
  = exit status 보존을 위해 일부 커널 자원 유지
  = ps에서 <defunct>로 표시

  문제: Zombie가 누적되면 커널의 프로세스 테이블이 고갈
        → 새 프로세스 생성 불가 → 시스템 장애

  해결: 부모가 wait() 또는 waitpid()로 Zombie 회수

Orphan Process:
  = 부모가 먼저 종료된 자식 프로세스
  = init(PID=1)이 자동으로 입양 → init이 주기적으로 wait() 호출

  문제: (상대적으로 덜 심각) init이 관리하므로 Zombie 누적 없음
  실제 장기 실행 서버: 명시적 reaping 필요 (예: SIGCHLD 핸들러)
```

★ 핵심 문제 3: execve() 후 메모리 상태

```
Q. fork() 후 자식이 execve()를 호출하면 어떤 변화가 일어나는가?

A.
fork() 직후 자식:
  = 부모의 주소 공간 복사본 (COW)
  = 부모의 PID와 다른 새 PID
  = 부모의 fd 테이블 복사

execve("ls", args, env) 호출 후:
  1. "ls" 실행 파일 로드
  2. 새 페이지 테이블 초기화
  3. .text, .data, .bss 영역 설정 (Private demand-zero 또는 file-
backed)
  4. 스택, 힙 초기화
  5. %rip → ls의 entry point
  6. PID 유지 ← 중요!
  7. 열린 fd 유지 ← 이것이 I/O 리다이렉션의 핵심!
  8. 정상 반환 없음 (오류 시에만 -1 반환)
```

Ch10. Signals

핵심 개념

```
pnb = pending & ~blocked
→ 0이 아니면 가장 낮은 번호 시그널부터 처리

큐 없음: 같은 타입은 최대 1개만 pending
         → 여러 번 전송해도 1번만 처리됨

Implicit Blocking:
  핸들러 실행 중 → 같은 타입 시그널 자동 차단
  핸들러 종료 시 → 차단 해제

async-signal-safe: write, kill, _exit ✓
                   printf, malloc, sprintf ✗ (내부 락 사용)
```

★ 핵심 문제 1: SIGCHLD 핸들러에서 while 루프가 필요한 이유

```
Q. SIGCHLD 핸들러에서 while 루프와 WNOHANG을 사용하는 이유는?

void sigchld_handler(int sig) {
    while (waitpid(-1, NULL, WNOHANG) > 0) ;  // ← while 필요!
}

A.
시그널 큐가 없기 때문.

상황: 3개의 자식이 거의 동시에 종료
→ 커널이 부모에게 SIGCHLD 전송
→ 하지만 pending bit는 1개뿐 (큐 없음)
→ SIGCHLD가 1번만 pending → 핸들러 1번만 실행

if 사용 시 (잘못된 코드):
  void sigchld_handler(int sig) {
      waitpid(-1, NULL, WNOHANG);  // 1개만 회수
  }
  → 나머지 2개 zombie 누적!

while + WNOHANG 사용 (올바른 코드):
```

```
  while (waitpid(-1, NULL, WNOHANG) > 0) ;
  → 한 번의 핸들러 호출에서 모든 terminated 자식 회수
  → WNOHANG: 종료된 자식 없으면 즉시 반환 (블로킹 없음)
```

★ 핵심 문제 2: 시그널 핸들러 안전성

```
Q. 다음 시그널 핸들러의 문제점은?

void handler(int sig) {
    char *err_msg;
    err_msg = (char *)malloc(24);    // 문제 1
    strcpy(err_msg, "error caught");
    printf("Handler: %s\n", err_msg); // 문제 2
}

A.
문제 1: malloc은 async-signal-safe가 아님
  → 메인 코드가 malloc 실행 중에 핸들러가 malloc 호출
  → malloc 내부의 free list가 불일치 상태일 때 핸들러가 개입
  → 힙 손상 또는 무한 루프

문제 2: printf는 async-signal-safe가 아님
  → 메인 코드가 printf 실행 중 (내부 락 보유)
  → 핸들러가 printf 호출 → 같은 락 요청
  → Deadlock!

올바른 패턴:
  volatile sig_atomic_t flag = 0;
  void handler(int sig) {
      flag = 1;  // 플래그만 설정 (async-signal-safe)
  }
  // 메인 루프에서 flag 확인하여 처리
```

Ch11. Threads

핵심 개념

```
스레드 전용: 레지스터(PC 포함), TID, 스택
  ⋆ 스택은 VM 보호 없음 → 포인터로 타 스레드 스택 접근 가능

스레드 공유: 코드(.text), 전역/static 데이터, 힙, fd 테이블

생성 비용: 프로세스 ~20K cycles, 스레드 ~10K cycles
구조: 피어(peer) 풀 (프로세스의 parent-child 트리와 다름)
```

공유 변수 판별 기준:

```
변수 x가 공유됨 ↔ 여러 스레드가 x의 어떤 인스턴스를 참조

Global: 1개 인스턴스 → 항상 공유
Local static: 1개 인스턴스 → 항상 공유
Local (non-static): 스레드별 스택에 각자 → 공유 아님
```

★ 핵심 문제 1: 스레드 인자 전달 Race Condition

```
Q. 다음 코드의 문제점과 해결 방법은?

for (int i = 0; i < N; i++)
    pthread_create(&tids[i], NULL, thread, (void*)&i);  // 잘못됨
```

```
void *thread(void *vargp) {
    int myid = *((int*)vargp);  // &i 역참조
    printf("Hello from thread %d\n", myid);
}

A. 문제: Race Condition
  모든 스레드가 동일한 &i를 가리킴
  main 루프가 i를 증가시키면 이미 생성된 스레드의 *vargp도 변함
  → 여러 스레드가 같은 myid를 출력하거나 N을 출력하는 등 오동작

해결 방법 1: malloc으로 각자 복사본 생성
  long *p = malloc(sizeof(long));
  *p = i;
  pthread_create(&tids[i], NULL, thread, (void*)p);
  // 스레드 내에서 free(vargp)

해결 방법 2: 정수 캐스팅 (포인터 크기 ≥ int 크기 보장 시)
  pthread_create(&tids[i], NULL, thread, (void*)(long)i);
  // 스레드 내에서: int myid = (int)(long)vargp;
```

Ch12. Synchronization

핵심 개념

```
cnt++ Race:
  L (load), U (update), S (store) 3단계가 비원자적
  → 두 스레드가 인터리빙되면 업데이트 소실

Mutex: test_and_set 원자적 연산으로 구현
  lock 비용: ~18배 성능 저하 (goodmcnt vs badcnt 실험)

Semaphore: s ≥ 0 불변
  wait(P): s>0이면 s--, 아니면 대기
  post(V): s++, 대기 중 스레드 하나 깨움

Deadlock 4조건: ME + Hold&Wait + No Preemption + Circular Wait
→ 회피: 모든 스레드가 동일 순서로 락 획득
```

★ 핵심 문제 1: Race Condition 추적

```
Q. 두 스레드가 각각 cnt++를 1번씩 수행한다. cnt 초기값=0일 때
   다음 인터리빙에서 최종 cnt 값은?

Thread 1: L1 → U1 → H2 → L2 → U2 → S2 → S1

A. 단계별 추적:
   L1: %rdx1 = cnt = 0
   U1: %rdx1 = 1 (아직 저장 안함)
   H2: Thread 2 시작 (cnt는 아직 0)
   L2: %rdx2 = cnt = 0
   U2: %rdx2 = 1
   S2: cnt = %rdx2 = 1
   S1: cnt = %rdx1 = 1  ← Thread 1의 업데이트로 덮어씀!

   최종 cnt = 1 (올바른 값은 2)
   → Thread 1의 업데이트가 소실됨 (lost update)
```

★ 핵심 문제 2: Producer-Consumer Semaphore 추적

```
Q. 1-element buffer에서 초기 empty=1, full=0일 때
```

```
   다음 실행 순서를 단계별로 추적하라:
   P1: wait(empty), buf=10, post(full)
   C1: wait(full), item=buf, post(empty)
   P2: wait(empty), buf=20, post(full)
   C2: wait(full), item=buf, post(empty)

A.
   초기: empty=1, full=0

   P1 wait(empty): empty: 1→0 (통과), buf = 10
   P1 post(full):  full: 0→1

   C1 wait(full):  full: 1→0 (통과), item = buf = 10
   C1 post(empty): empty: 0→1

   P2 wait(empty): empty: 1→0 (통과), buf = 20
   P2 post(full):  full: 0→1

   C2 wait(full):  full: 1→0 (통과), item = buf = 20
   C2 post(empty): empty: 0→1

   결과: C1이 10을, C2가 20을 소비 (올바름)
   만약 P1 post 전에 C1이 wait하면: C1 블로킹 → P1 post → C1 진행
```

★ 핵심 문제 3: Deadlock 분석과 회피

```
Q. 다음 코드에서 Deadlock이 발생하는 시나리오를 설명하고 해결책을 제시하라.

void *count(void *vargp) {
    int id = (int)vargp;
    wait(&mutex[id]);
    wait(&mutex[1-id]);  // id=0이면 mutex[1], id=1이면 mutex[0]
    cnt++;
    post(&mutex[id]); post(&mutex[1-id]);
}

A. Deadlock 시나리오:
   Thread 0: wait(mutex[0]) 성공 (mutex[0]=0)
   Thread 1: wait(mutex[1]) 성공 (mutex[1]=0)
   Thread 0: wait(mutex[1]) → mutex[1]=0 → 영원히 대기
   Thread 1: wait(mutex[0]) → mutex[0]=0 → 영원히 대기
   → Circular Wait → Deadlock!

   Deadlock 4조건 확인:
   ✓ ME: mutex는 하나씩만 보유 가능
   ✓ Hold&Wait: mutex[0] 보유하면서 mutex[1] 대기
   ✓ No Preemption: mutex는 자발적으로만 반납
   ✓ Circular Wait: T0→mutex[1]→T1→mutex[0]→T0

   해결: 동일 순서로 락 획득 (Circular Wait 제거)
   wait(&mutex[0]); wait(&mutex[1]);  // 모든 스레드가 동일 순서
   → T1도 mutex[0] 먼저 시도 → T0가 보유 중이면 대기
   → T0가 완료 후 해제 → T1 진행 → Deadlock 없음
```

★ 핵심 문제 4: Thread Safety 분류

```
Q. 다음 함수들을 Thread Safety Class로 분류하라.
   (Class 1: 공유변수 미보호, Class 2: 상태 유지, Class 3: unsafe 함수 호
출)

(a) int rand() { next = next*C + D; return next/E % F; }  (next는
```

```
static)
(b) char *strtok(char *s, const char *d);  (내부에 static 버퍼 사용)
(c) void bad_cnt() { cnt++; }  (cnt는 전역, 보호 없음)

A.
(a) Class 2: static 변수 next에 상태 저장 → 스레드 간 경쟁
    해결: rand_r(unsigned int *nextp) — 상태를 인자로 전달

(b) Class 2: 내부 static 버퍼에 상태 저장
    해결: strtok_r(char *s, const char *d, char **saveptr)

(c) Class 1: 공유 변수 cnt를 보호 없이 접근
    해결: mutex로 cnt++ 보호
```

Ch13. System-Level I/O

핵심 개념

```
Everything is a file: 디스크, 터미널, 소켓, 커널(/proc) 모두 파일

커널 파일 표현 3 테이블:
  Descriptor Table (프로세스별): fd번호 → Open File Table 항목
  Open File Table (전역 공유): file pos + refcnt + v-node 포인터
  v-node Table (전역 공유): 실제 파일 메타데이터

Short Count: read/write가 요청보다 적게 처리 → 오류 아님!
  발생: EOF, 터미널 입력, 네트워크 소켓

I/O 함수 선택:
  디스크/터미널: Standard I/O (버퍼링 자동 처리)
  시그널 핸들러: Unix I/O (async-signal-safe)
  소켓: RIO (Short count 처리)
  소켓 + Standard I/O: ✖ 금지
```

★ 핵심 문제 1: fork() 후 파일 공유

```
Q. 다음 코드 실행 후 파일 디스크립터 상태를 설명하라.
   fd = open("file.txt", O_RDONLY);
   read(fd, &c1, 1);  // 'a' 읽음, pos=1
   fork();
   // 자식에서 read(fd, &c2, 1) 호출

A.
fork() 전:
  부모 DT: fd → OFT[A] (pos=1, refcnt=1)

fork() 후:
  부모 DT: fd → OFT[A] (pos=1, refcnt=2)  ← 공유!
  자식 DT:  fd → OFT[A] (pos=1, refcnt=2)  ← 동일 OFT 항목

자식이 read(fd, &c2, 1):
  OFT[A].pos: 1→2
  c2 = 'b' (pos=1에서 읽음)

부모가 이후 read(fd, &c3, 1):
  OFT[A].pos: 2→3  (자식이 이미 pos를 2로 변경!)
  c3 = 'c'

★ 핵심: fork 후 부모와 자식이 file pos를 공유
```

```
  → 한쪽의 read가 다른 쪽의 읽기 위치에 영향을 줌
  → 의도치 않은 동작의 원인이 될 수 있음
```

★ 핵심 문제 2: dup2() I/O 리다이렉션

```
Q. "ls > foo.txt" 명령을 구현하는 코드를 작성하고 각 단계의 fd 상태를 설명하라.

A.
단계별 구현:
  // 1. fork()
  if ((pid = fork()) == 0) {
      // 자식 프로세스에서:

      // 2. foo.txt 열기 → fd=3 할당 (0,1,2는 이미 사용)
      int fd = open("foo.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666);
      // fd 3 → OFT[B] (foo.txt, pos=0, refcnt=1)
      // fd 1 → OFT[A] (terminal, pos=0, refcnt=2) ← 부모와 공유

      // 3. dup2(fd, 1): fd1(stdout)이 foo.txt를 가리키게 함
      dup2(fd, 1);
      // dup2 동작:
      //   fd 1이 가리키던 OFT[A] 닫음 (refcnt: 2→1)
      //   fd 1 → OFT[B] (foo.txt) 연결 (refcnt: 1→2)
      // fd 1 → OFT[B], fd 3 → OFT[B]

      // 4. fd=3 닫기 (더 이상 불필요)
      close(fd);
      // fd 3 → NULL (OFT[B] refcnt: 2→1)
      // fd 1 → OFT[B]  ← stdout이 foo.txt 가리킴

      // 5. ls 실행 → ls의 stdout(fd=1)이 foo.txt에 쓰임
      execve("/usr/bin/ls", args, environ);
  }
```

★ 핵심 문제 3: Standard I/O 버퍼링과 fork()

```
Q. 다음 코드의 출력 결과는? (줄바꿈 없는 printf 주의)

int main() {
    printf("hello");  // '\n' 없음 → 버퍼에 저장
    fork();
    // 부모와 자식 모두 exit()
}

A.
printf("hello"): '\n'이 없으므로 Standard I/O 버퍼에 저장
  → 실제 write() 시스템 콜 호출 안됨

fork() 후:
  → 부모의 Standard I/O 버퍼("hello")가 자식에게도 복사됨!

프로세스 종료 시(exit()):
  → Standard I/O 버퍼 flush → write("hello") 호출

결과: "hello"가 2번 출력!
  (부모 1번 + 자식 1번)

해결: fork() 전에 fflush(stdout) 또는 '\n' 사용
  printf("hello\n");  // '\n'으로 즉시 flush
  fork();             // 버퍼 비워진 후 fork
```

```
  → "hello"가 1번만 출력
```

Ch14. Debugger & ptrace

핵심 개념

```
ptrace: tracer(GDB)가 tracee(디버깅 대상)의 메모리/레지스터 완전 제어
API: long ptrace(request, pid, addr, data)

GDB 동작 흐름:
  fork() → 자식: PTRACE_TRACEME → execve() → SIGTRAP(자동) → 정지
  부모(GDB): waitpid() → PTRACE_POKETEXT(0xcc) → PTRACE_CONT
  → 브레이크 도달 → SIGTRAP → PTRACE_GETREGS → 복원 → PTRACE_SINGLESTEP

strace: PTRACE_SYSCALL로 syscall 진입/탈출 추적
```

Breakpoint 3종:

```
Software: int 3(0xcc) 삽입 → SIGTRAP / 무제한 / 코드 수정 필요
Hardware: DR0~DR3 레지스터 / 최대 4개 / 코드 수정 불필요
Memory:   페이지 권한 변경 → Protection Fault / 읽기/쓰기 접근 감지
```

★ 핵심 문제 1: Software Breakpoint 설정/해제 절차

```
Q. GDB가 주소 0x400500에 Software Breakpoint를 설정하고
   실행 후 계속 진행할 때의 전체 절차를 설명하라.

A.
[설정]
  1. orig = PTRACE_PEEKTEXT(pid, 0x400500)  // 원래 바이트 저장
  2. PTRACE_POKETEXT(pid, 0x400500, (orig & ~0xff) | 0xcc)
     // 첫 바이트를 0xcc로 교체
  3. PTRACE_CONT  // 실행 재개

[실행 중]
  4. CPU가 0x400500 도달 → 0xcc 실행
  5. CPU: SIGTRAP 예외 발생 → 프로세스 정지
  6. GDB: waitpid() 반환

[상태 확인 및 계속 진행]
  7. PTRACE_GETREGS  // 현재 레지스터 값 확인
     (이 때 %rip = 0x400501, 0xcc 다음 주소)
  8. PTRACE_POKETEXT(pid, 0x400500, orig)  // 원래 바이트 복원
  9. PTRACE_SETREGS (rip = 0x400500)       // PC를 브레이크 주소로 되돌
림
  10. PTRACE_SINGLESTEP  // 0x400500의 원래 명령어 1개 실행
  11. waitpid() → SIGTRAP
  12. PTRACE_POKETEXT(pid, 0x400500, 0xcc)  // 브레이크포인트 재설정
  13. PTRACE_CONT  // 계속 실행
```

★ 핵심 문제 2: ptrace Request 구분

```
Q. 다음 각 상황에서 어떤 ptrace request를 사용해야 하는가?

(a) tracee의 특정 주소에서 값을 읽기
(b) tracee의 %rax 값을 수정하기
(c) 다음 syscall이 실행될 때까지 tracee를 실행
(d) 명령어 1개만 실행하고 정지
(e) 자신을 디버깅 받도록 설정 (자식 초기화 시)
```

```
A.
(a) PTRACE_PEEKTEXT 또는 PTRACE_PEEKDATA
(b) PTRACE_SETREGS (전체 레지스터 구조체 덮어쓰기)
(c) PTRACE_SYSCALL
(d) PTRACE_SINGLESTEP
(e) PTRACE_TRACEME
```

Ch15. Security Attacks

핵심 개념

```
Cold Boot:  DRAM 냉각(-50°C) → 0.2% 미만 손실 → 키 추출
            대응: TRESOR(키를 CPU 레지스터에만 유지)

Timing SC:  strcmp 조기 반환 → 일치 수에 비례 시간 증가
            복잡도: 52^9 → 52×9
            대응: constant-time 비교

Flush+Reload: clflush → victim 접근 대기 → reload 시간 측정
              cache hit = victim이 접근함

Meltdown:   Speculative Exec + cache covert-channel
            권한 검사 전 커널 주소 투기적 실행 → 캐시에 흔적
            대응: KPTI(커널/유저 PT 분리, 5~20% 저하)

Rowhammer:  인접 Row 반복 접근 → 비트 플립
            도전1: clflush로 cache bypass
            도전2: VA 하위 12bit = PA 하위 12bit → 인접 row 역공학
```

★ 핵심 문제 1: Timing Side-Channel 복잡도 계산

```
Q. 9자리 비밀번호 (대소문자 알파벳 52자 중)를 Timing Attack으로 크래킹할 때
   최대 시도 횟수는? 무작위 추측과 비교하라.

A.
무작위 추측:
  전체 경우의 수 = 52^9 ≈ 2.3 × 10^15
  평균 시도: 52^9 / 2 ≈ 10^15 → 사실상 불가능

Timing Attack (strcmp 조기 반환 이용):
  1번째 자리: 52가지 시도 → 시간이 가장 오래 걸리는 것 선택
  2번째 자리: 52가지 시도 (1번째 자리 고정)
  ...
  9번째 자리: 52가지 시도

  총 시도: 52 × 9 = 468 → 현실적으로 가능!

비교: 2.3×10^15 vs 468 → 약 5×10^12배 차이
```

★ 핵심 문제 2: Meltdown 공격 단계 분석

```
Q. 다음 Meltdown PoC 코드에서 각 단계를 설명하라.

char user_array[255 * 4096];
// (A) clflush로 user_array 전체 캐시 비우기
char secret = *(char *)0xffffffff81a000e0;  // (B)
maccess(user_array[secret * 4096]);          // (C)
// (D) for문으로 user_array 각 인덱스 접근 시간 측정

A.
```

```
(A) Flush 단계:
    user_array의 모든 캐시 라인을 clflush로 제거
    → 이후 어느 캐시 라인이 로드됐는지 구분 가능하게 준비

(B) 투기적 실행 (Transient Execution):
    0xffff...은 커널 주소 → 유저 접근 불가
    BUT: CPU는 권한 검사를 나중에 수행 (투기적으로 먼저 실행)
    → CPU가 실제로 커널 메모리에서 값을 읽어 secret에 저장
    → 이후 권한 위반으로 예외 처리, secret 레지스터 롤백

(C) Covert Channel (정보 전송):
    secret 값(0~254)을 user_array 인덱스로 사용
    → user_array[secret × 4096] 캐시 라인이 로드됨
    → 예외 처리 후에도 캐시 상태는 유지됨! ← 핵심

(D) Flush+Reload (정보 수신):
    각 index의 user_array[index×4096] 접근 시간 측정
    → 유일하게 빠른(cache hit) index = secret 값
    → 커널 메모리 값 복원 완료!

핵심 원리: 캐시 상태는 예외 처리 시 롤백되지 않음
```

★ 핵심 문제 3: Rowhammer 공격 단계

```
Q. Rowhammer로 Linux 커널 권한 상승을 달성하는 절차를 설명하라.

A.
목표: PTE(Page Table Entry)의 비트를 플립하여 커널 메모리 접근

도전 극복:
  #1 Cache bypass: clflush로 매번 캐시 비우기
     → 반복 접근이 DRAM에 직접 도달하여 Row 활성화 발생

  #2 Address info: VA 하위 12bit = PA 하위 12bit 이용
     → 대량 메모리 할당 후 주소 패턴 역공학
     → 인접 Physical Row에 해당하는 VA 파악

공격 단계:
  ① 대량 메모리 할당 (2GB 등)
  ② Rowhammer: 인접 Row A, B를 반복 접근 (clflush 포함)
     while(1): access(A); access(B); clflush(A); clflush(B);
  ③ Victim Row에서 비트 플립 발생 위치 탐색
  ④ 해당 메모리를 커널에 반환 (free)
  ⑤ 커널이 반환 메모리를 PTE 배열로 사용하도록 유도
     (대량의 mmap 호출로 PTE 확장 강제)
  ⑥ Rowhammer: 비트 플립 발생!
     - RW bit 플립(2% 확률): 쓰기 권한 획득
     - PPN 비트 플립(31% 확률): 다른 물리 페이지(커널) 접근
  ⑦ 커널 물리 페이지에 직접 읽기/쓰기 → 권한 상승 완료
```